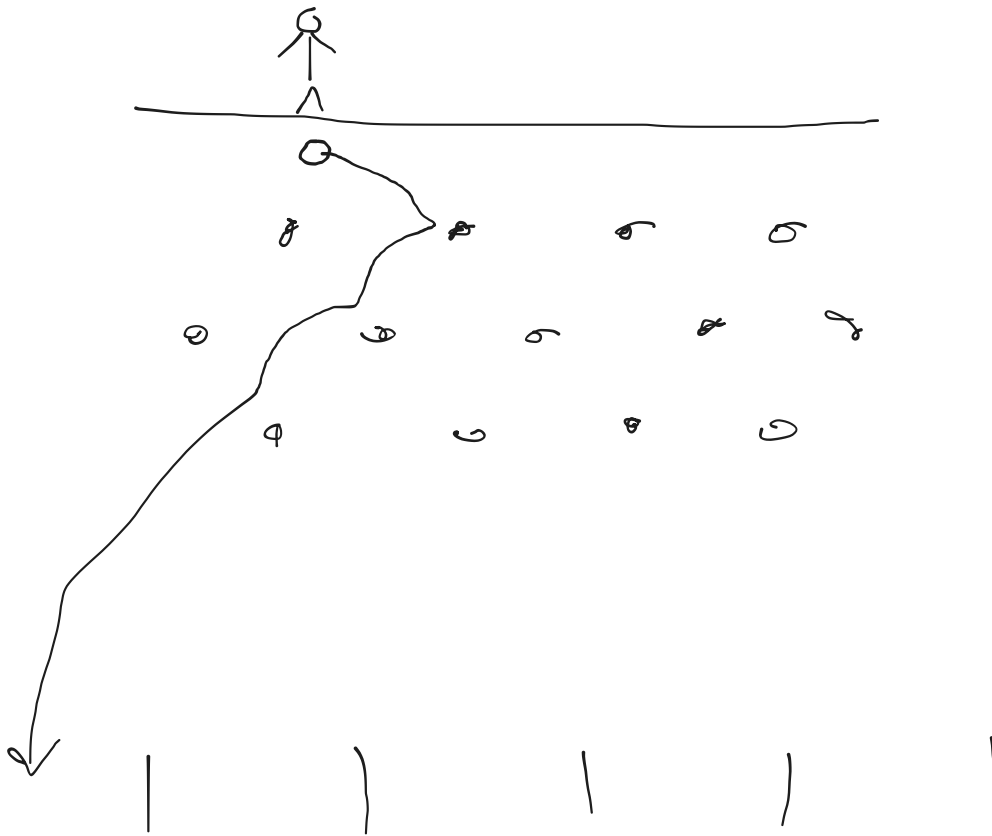


Physicsbodies

PhysicsBodies

Picture a game where your player moves side to side at the top of the screen, drops a coin, and that coin bounces on small pegs below before falling into a pit at the bottom of the screen:



This scenario has a complex set of interactions between the objects listed. The player needs to be predictably and easily controlled, the ball needs to be able to move on its own following a realistic physics simulation and bounce off the pegs, and the pegs need to be able to remain stationary when they are struck by the ball. To code all of these interactions ourselves would be quite the task. Luckily, Godot has a family of Nodes that cover these sorts of physics interactions, the `PhysicsBodies`.

We use the `PhysicsBodies` whenever we want to create a scene or element that will interact with the *physics* of our game. This could be the player, the ground, walls, ceilings, enemies,

projectiles, platforms, and countless other examples. Godot has three PhysicsBodies that cover all of these uses:

- StaticBody2D
- RigidBody2D
- CharacterBody2D

Each PhysicsBody has a particular use, let's run through them.

StaticBody2D

The StaticBody2D is the simplest of all of the PhysicsBodies. We use StaticBodies for any object that will not move that we want our other PhysicsBodies to *collide* with. Here's some examples of things we might use a StaticBody2D for:

- Floors
- Walls
- Ceilings
- Platforms

RigidBody2D

The RigidBody2D is used for objects that the player will not directly control, but that we want a physics simulation applied to. RigidBodies will move on their own following the laws of physics (or what we define as the laws of physics for our game world) without the player's direct interaction. This means they will fall, bounce, roll, and shift all on their own! Some examples of things we might use a RigidBody2D for include:

- A ball
- An arrow
- Falling rocks
- A box the player has to push

CharacterBody2D

CharacterBody2D is an extremely useful tool, but it requires us to do some extra coding work. The CharacterBody2D Node (often called a Kinematic Body in other engines) is used when we want our player to have direct control over an object with a *predictable* physics simulation done to it. This means that in order to move a CharacterBody2D we have to directly set its *velocity* property and use its `move_and_slide()` method to move it. Here's an example:

```
# Using a Node reference
@export var my_character_body : CharacterBody2D = $MyCharacterBody

func _physics_process(delta: float) -> void:
    # This sets the MyCharacterBody's velocity to move 20 pixels/s to the
    right
    my_character_body.velocity = Vector2(20, 0)
    my_character_body.move_and_slide()
```

or

```
# Using a direct control. Remember, the script must be attached to the
CharacterBody2D we are wanting to control if we do it this way.

func _physics_process(delta: float) -> void:
    # This sets the MyCharacterBody's velocity to move 20 pixels/s to the
    right
    velocity = Vector2(20, 0)
    move_and_slide()
```

Typically, we will start from the CharacterBody2D's default script template, CharacterBody2D: Basic Movement. This script handles the majority of typical movement we would want for a character (running, jumping, falling). Here it is below:

```
extends CharacterBody2D

const SPEED = 300.0
const JUMP_VELOCITY = -400.0

func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Get the input direction and handle the movement/deceleration.
    # As good practice, you should replace UI actions with custom gameplay
    actions.
    var direction := Input.get_axis("ui_left", "ui_right")
```

```
if direction:
    velocity.x = direction * SPEED
else:
    velocity.x = move_toward(velocity.x, 0, SPEED)

move_and_slide()
```

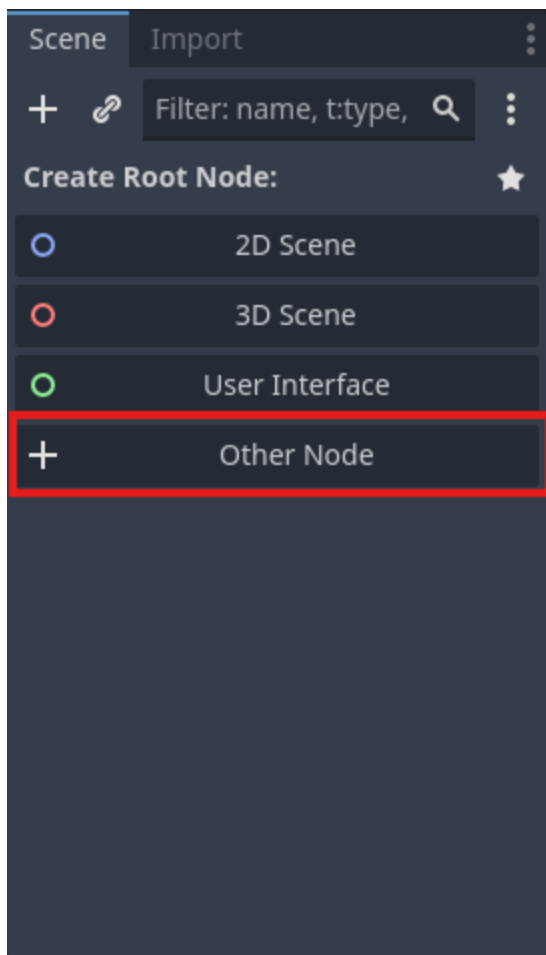
In addition, RigidBody2Ds may not interact exactly how we expect them to (by default we are not able to push a RigidBody2D with a CharacterBody2D). This is because CharacterBody2Ds are used for *predictable* physics simulations. This means we must tell the CharacterBody2D how we expect it to interact with the physics simulation. Don't worry, this doesn't mean we'll need to re-write the physics engine itself, just keep in mind that if something doesn't work as expected we can adjust it in code!

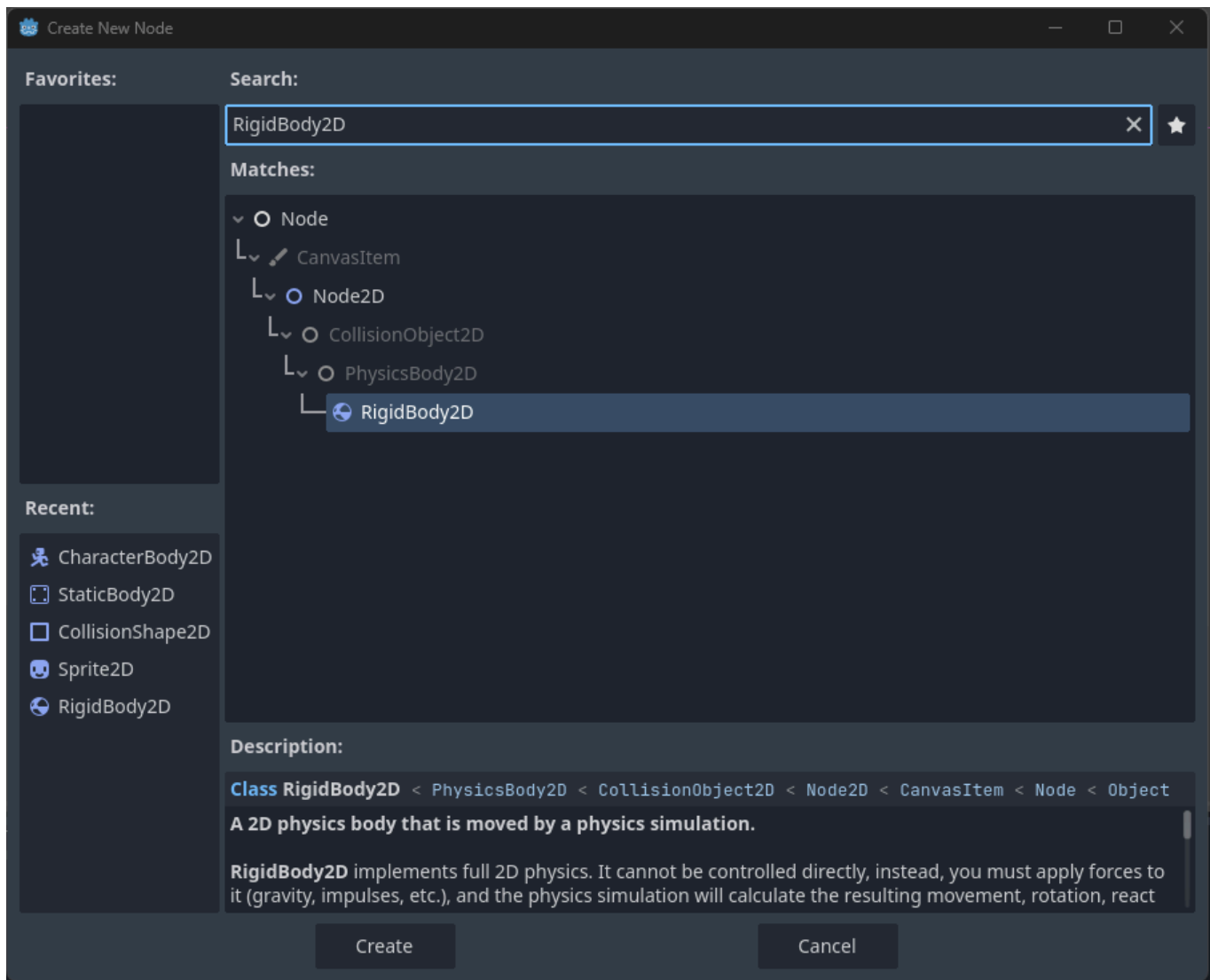
Here's some examples of uses for a CharacterBody2D:

- The player's character
- An enemy
- A creature that follows the player around
- An NPC that walks around town

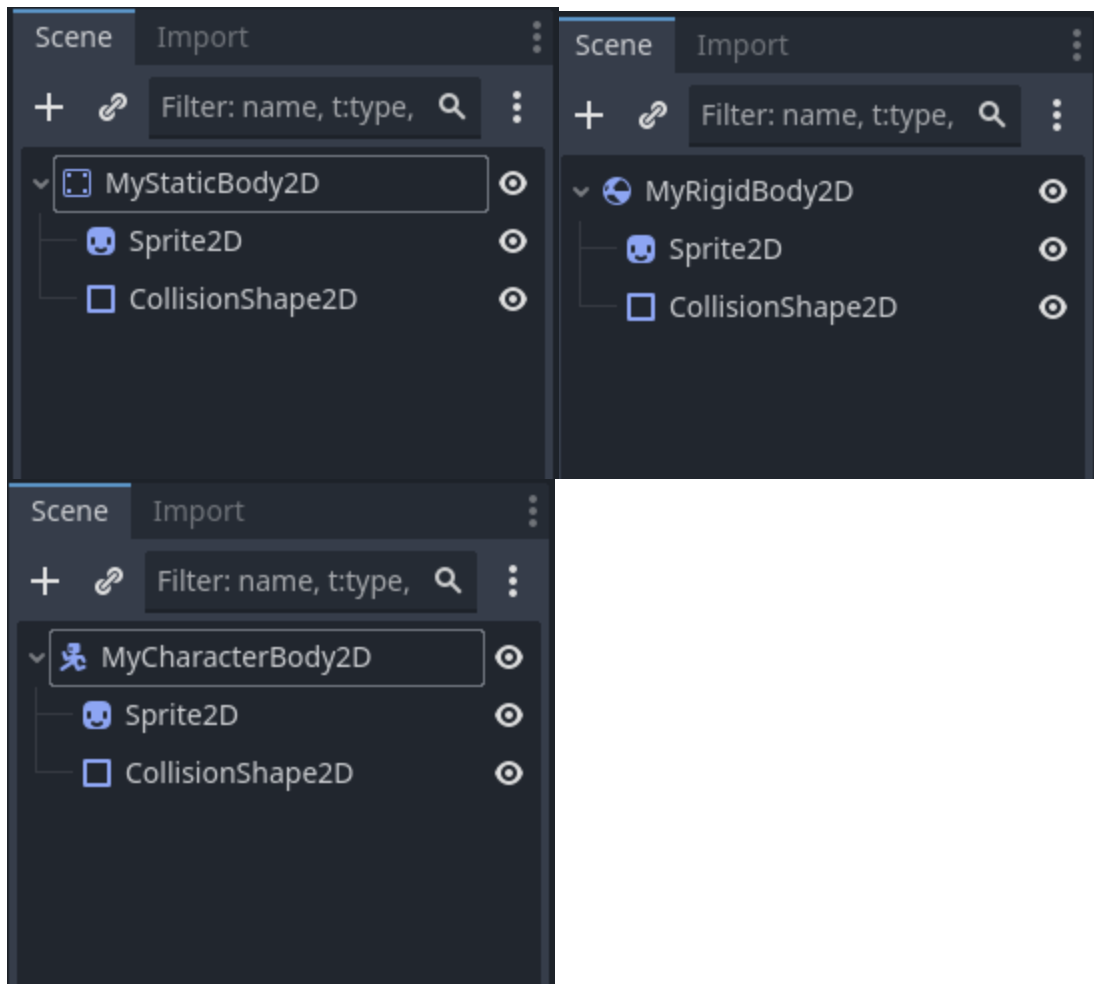
Anatomy of a PhysicsBody

Typically, we make any PhysicsBody that moves into its own scene (most often RigidBody2Ds or CharacterBody2Ds). This means the root node of the scene will be the PhysicsBody itself, not a Node2D. We can do this by selecting the "Other Node" option when creating a new scene and selecting the type of PhysicsBody we want to use!

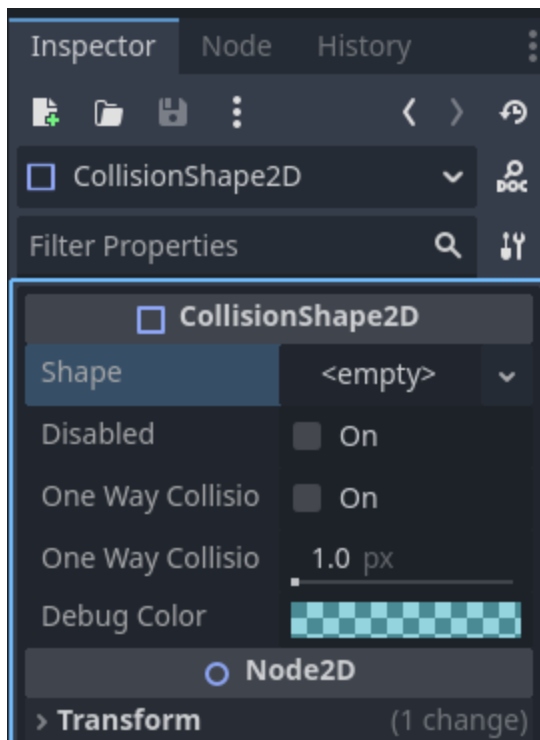




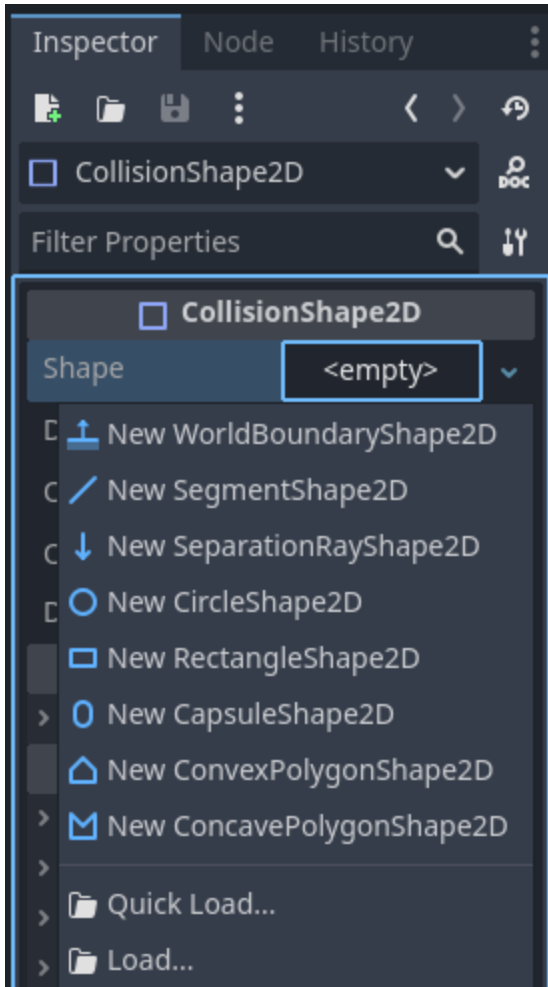
All PhysicsBodies need more than just themselves to function. A PhysicsBody only tells Godot that the object we are making will interact with physics in a specific way. In order to properly configure a PhysicsBody we need to tell Godot the *shape* of the object and what that object will *look* like. We've already discussed the Node used for displaying images, Sprite2D. The Node used for defining the space, or shape, a PhysicsBody takes up is called a CollisionShape2D. The CollisionShape2D we use for a PhysicsBody *must* be a direct child of the PhysicsBody, otherwise the PhysicsBody will not work. Here's an example of what a properly configured Scene Tree for all three PhysicsBodies would look like:



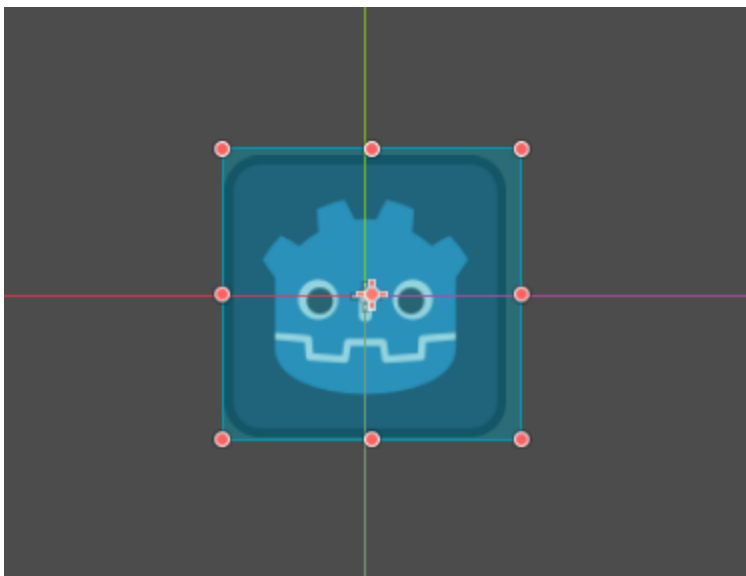
Note that all three PhysicsBodies' Scene Trees look the same when properly configured. Next we will need to define the exact *shape* of the CollisionShape2D. We will do this in the inspector by changing the CollisionShape2D's *shape* property:



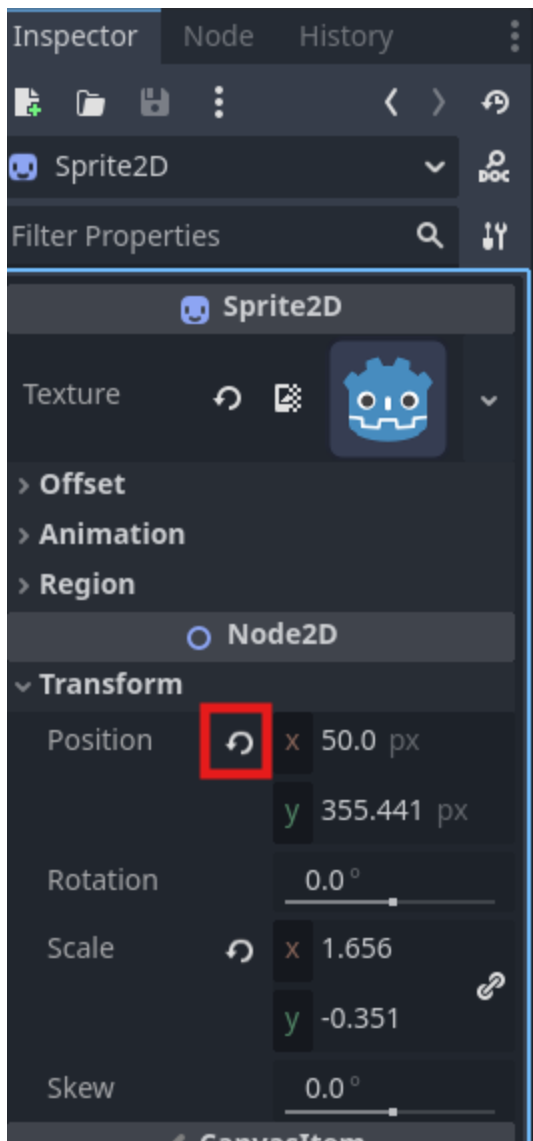
Clicking on the "empty" dropdown menu will give us some options of what *kind* of shape we want to use:



We'll typically use Rectangle and Capsule shapes the most, but all of the given shapes can be useful! Once we've selected a Shape we can align it whatever image we've chosen for our Sprite2D. Here's what that would look like for a RectangleShape and a Sprite2D using the Godot icon:



Note: It's important that we do not move the Sprite2D or CollisionShape2D off of the *origin* of the scene. If you do accidentally move them you can reset their position by clicking the reset button on their position property in the transform section in the inspector.



Now it's your turn!

For this assignment you will make the following scenes:

- Scene 1: A scene that is a small level with PhysicsBodies placed to stop the player from exiting the screen from the sides and bottom, but not the top.
 - Hint: The root node of this scene will be a Node2D that you will rename "Main"
- Scene 2: A scene that is a PhysicsBody that can be controlled by the player
 - Look online for an image of something that would catch a falling object
- Scenes 3-5: 3 scenes that are a PhysicsBody that will fall on its own
 - Look online for 3 different images that our player would catch

Once you have completed all scenes, place Scene 2 in Scene 1. Then place Scenes 3-5 in Scene 1 *above* the player.